ANALYSE PRÉLIMINAIRE — APPRENTISSAGE PAR RENFORCEMENT SUR *FrozenLake*

A PREPRINT

Ibrahim R. HADJ-ARAB
Faculté des sciences et de génie
Université Laval
ibrahim-rabah.hadj-arab.1@ulaval.ca

Khalil EN NOUAARY
Faculté des sciences et de génie
Université Laval
khalil.en-nouaary.1@ulaval.ca

30 mars 2026

Code source disponible à : `https://github.com/Ribraba/IFT-7201-Projet`

1 Cas d'étude

1.1 Composantes de l'environnement

FrozenLake-v1 [Towers et al., 2024] est un environnement de navigation en grille $N \times N$. L'espace d'états est l'ensemble des N^2 cases ; l'espace d'actions contient quatre déplacements (haut, bas, gauche, droite). Trois composantes peuvent être manipulées pour moduler la difficulté de l'environnement et l'exposition aux situations dangereuses :

- **Degré de stochasticité.** Le paramètre `is_slippery` contrôle la dynamique de transition. Lorsque activé, chaque action entraîne le déplacement souhaité avec probabilité $1/3$ et l'un des deux déplacements perpendiculaires avec probabilité $1/3$ chacun, rendant l'environnement non-déterministe.
- **Nombre de situations dangereuses.** Les cases de type `H` (trous) constituent les situations dangereuses. Augmenter leur nombre réduit la surface praticable et multiplie les risques de chute.
- **Positionnement des situations dangereuses.** Regrouper les trous de façon contiguë (ex. rangée horizontale) crée un obstacle de type falaise, forçant l'agent à choisir entre un chemin court risqué et un détour sûr.

La fonction de récompense est mise en forme pour accélérer la propagation du signal de danger : chute dans un trou ($r = -1{,}0$), atteinte de l'objectif ($r = +1{,}0$), collision avec un mur ($r = -0{,}1$), déplacement ordinaire ($r = 0{,}0$).

1.2 Cas d'étude

Le Tableau 1 et la Figure 1 présentent les trois configurations retenues.

TABLE 1 – Configurations des trois cas d'étude.

	Facile	**Moyen**	**Difficile**
Taille de la grille	4×4	4×4	6×6
`is_slippery`	Non	Oui	Oui
Nombre de trous	3	3	4
Positionnement	Colonne droite	Colonne droite	Rangée centrale
Position de l'objectif	Bas-centre	Bas-centre	Bas-gauche

Facile (4×4, déterministe, 3 trous en colonne). Les transitions sont déterministes. Ce cas sert de référence : en l'absence de bruit, toute stratégie TD doit converger vers un taux de chutes nul et un taux de boucles nul. Il valide l'implémentation et établit une borne supérieure de performance.

Moyen (4×4, stochastique, 3 trous en colonne). Topologie identique à *Facile* ; seul `is_slippery` change. Ce cas isole rigoureusement l'effet de la stochasticité : toute dégradation de performance est attribuable uniquement au non-déterminisme, non à la topologie de la grille.

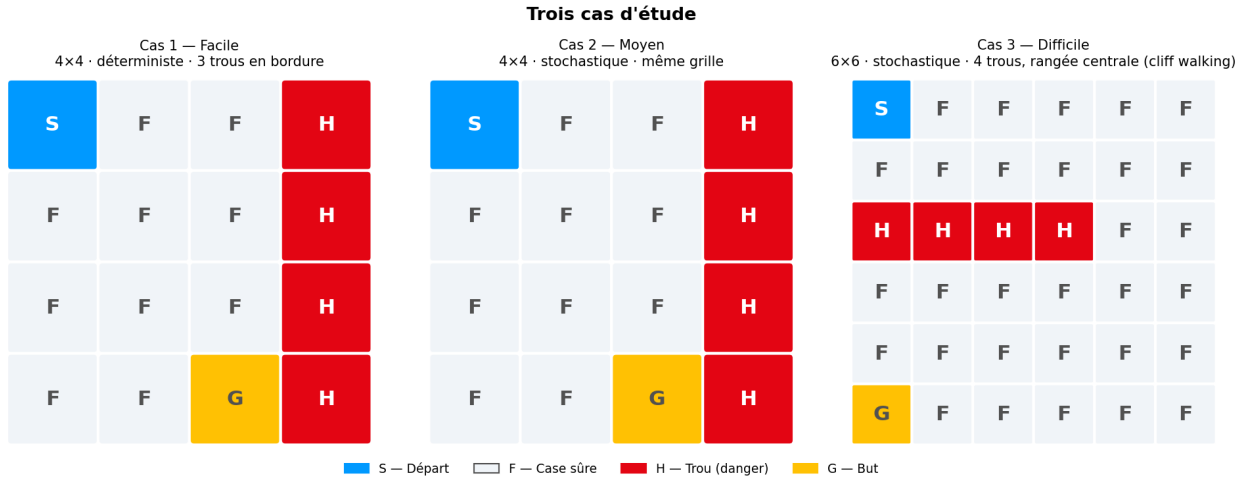


FIGURE 1 – Visualisation des trois environnements. `S` : départ, `G` : objectif, `H` : trou (situation dangereuse), `F` : case libre.

Difficile (6×6, stochastique, 4 trous contigus en rangée centrale, objectif en bas-gauche). La rangée `HHHHFF` crée une barrière horizontale analogue au problème du *cliff walking* [Sutton and Barto, 2018, chap. 6]. L'objectif étant placé en bas-gauche, directement au-dessous de la barrière, deux trajectoires coexistent : un chemin court risqué descendant par la colonne gauche (en bordure des trous) et un long détour sûr contournant la barrière par les colonnes droites. Le passage à la grille 6×6 est délibéré : il augmente l'espace d'états pour que le détour sûr soit significativement plus long que le chemin risqué, condition nécessaire pour que la divergence *off-policy*/*on-policy* soit observable et mesurable par l'algorithme de Dijkstra.

2 Stratégies considérées

Deux stratégies de différence temporelle (TD) sont sélectionnées comme baselines : Q-learning et SARSA. Ces méthodes ne disposent d'aucun mécanisme conçu explicitement pour éviter les situations dangereuses, ce qui permet d'établir une limite de référence que toute stratégie de sécurité ultérieure devra dépasser. Le choix de méthodes tabulaires (plutôt que DQN [Mnih et al., 2015] ou PPO) est justifié par la taille réduite de l'espace d'états ($4 \times 4 = 16$ et $6 \times 6 = 36$ états) : les méthodes tabulaires convergent vers la solution exacte sans approximation de fonction, ce qui garantit que les différences observées reflètent bien les propriétés intrinsèques des algorithmes. De plus, Q-learning et SARSA forment la paire canonique *off-policy* / *on-policy* dont les comportements divergents face aux obstacles sont directement prédits par la théorie [Sutton and Barto, 2018].

Q-learning [Watkins and Dayan, 1992] est une méthode *off-policy* dont la règle de mise à jour repose sur la valeur maximale de l'état suivant :

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]. \tag{1}$$

Cette mise à jour optimiste suppose que la politique optimale sera suivie dans le futur, indépendamment de l'exploration en cours. Dans des environnements à obstacles groupés, cette hypothèse peut conduire à sous-estimer le risque réel de chute et à préférer le chemin court risqué.

SARSA [Rummery and Niranjan, 1994] est une méthode *on-policy* qui utilise l'action réellement sélectionnée selon la politique ε-greedy courante :

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma\, Q(s',a') - Q(s,a)\big]. \tag{2}$$

Puisque a' peut être une action aléatoire (avec probabilité ε), SARSA intègre le coût de l'exploration dans ses estimations. On s'attend donc à ce qu'il apprenne à éviter davantage les cases proches des trous, au prix d'une propagation plus lente du signal de danger en début d'apprentissage. La confrontation de ces deux comportements opposés face au risque est la principale motivation de leur sélection.

3 Méthodologie expérimentale

Librairies. L'environnement est fourni par Gymnasium 1.2.3 [Towers et al., 2024]. Q-learning et SARSA sont implémentés directement en Python/NumPy 2.2.6, sans recours à Stable-Baselines3 [Raffin et al., 2021], afin de contrôler précisément les mises à jour TD.

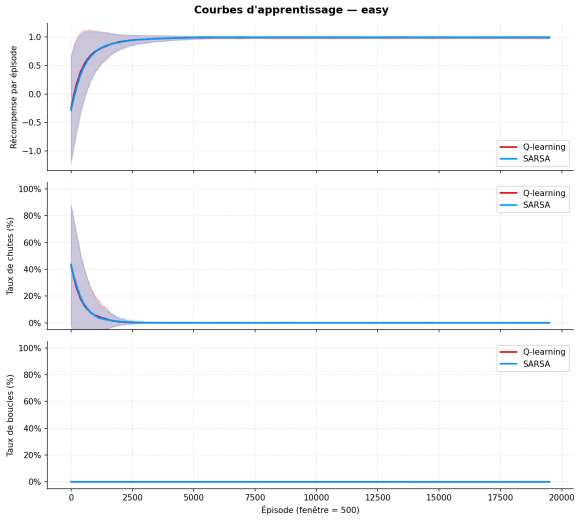
Hyper-paramètres. Les deux algorithmes partagent : $\gamma = 0{,}99$ (horizon long, adapté aux trajectoires multi-étapes jusqu'à l'objectif), $\alpha_0 = 0{,}5$ (valeur standard pour les méthodes TD tabulaires [Sutton and Barto, 2018]), décroissance $0{,}9995$ pour α et ε jusqu'aux minima $\alpha_{\min} = 0{,}01$ et $\varepsilon_{\min} = 0{,}05$. Ce taux de décroissance amène ε à son minimum après $\approx 6\,000$ épisodes, assurant une phase d'exploration large suivie d'une exploitation progressive sur les 14 000 épisodes restants.

Paramètres expérimentaux. Chaque configuration est entraînée sur 20 000 épisodes, avec un maximum de 200 pas par épisode. Les expériences sont répétées avec $n = 10$ graines indépendantes (graines 0 à 9) pour produire des intervalles de confiance (moyenne $\pm$ écart-type). L'évaluation finale utilise la politique greedy ($\varepsilon = 0$) sur 500 épisodes.

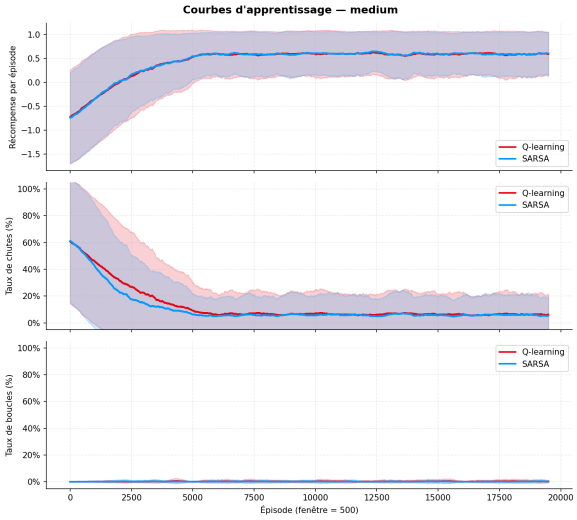
Indicateurs de performance. L'étude évalue la sécurité *durant* l'apprentissage et sur la politique finale. Cinq indicateurs sont mesurés :

- **Récompense cumulée durant l'entraînement** : indicateur de convergence de la politique (moyenne mobile, fenêtre 500 épisodes).
- **Taux de chutes durant l'entraînement** : fraction d'épisodes terminés dans un trou. Mesure la sécurité au cours de l'apprentissage.
- **Taux de boucles durant l'entraînement** : fraction d'épisodes terminés par dépassement du nombre maximal de pas. Distingue l'échec par danger (chute) de l'échec par inefficacité (blocage) — un agent qui évite les trous mais reste circulaire n'est pas opérationnel.
- **Taux d'états dangereux à l'évaluation** : fraction de pas passés dans une case adjacente à un trou lors de l'évaluation greedy ($\varepsilon = 0$). Quantifie la proximité aux obstacles de la politique finale.
- **Taux de chemins sécurisés à l'évaluation** : fraction des épisodes réussis dont la trajectoire chevauche à au moins 50 % le chemin optimal de sécurité, calculé par une variante de Dijkstra [Dijkstra, 1959] pénalisant les cases adjacentes à un trou (cas Difficile uniquement). Quantifie la capacité de la politique finale à emprunter le détour sûr.

4 Résultats



(a) **Facile** — Convergence vers 0 % de chutes et 0 % de boucles dès $\approx 2\,000$ épisodes. Valide les deux implémentations en environnement déterministe.



(b) **Moyen** — La stochasticité fait bondir le taux de chutes initial. Le taux ne converge pas vers zéro : $\varepsilon_{\min} = 5\%$ combiné aux glissades maintient un risque irréductible lié au bruit de transition.

FIGURE 2 – Courbes d'apprentissage (3 panneaux : récompense, chutes, boucles). Bande colorée : $\pm$ un écart-type sur 10 répétitions.

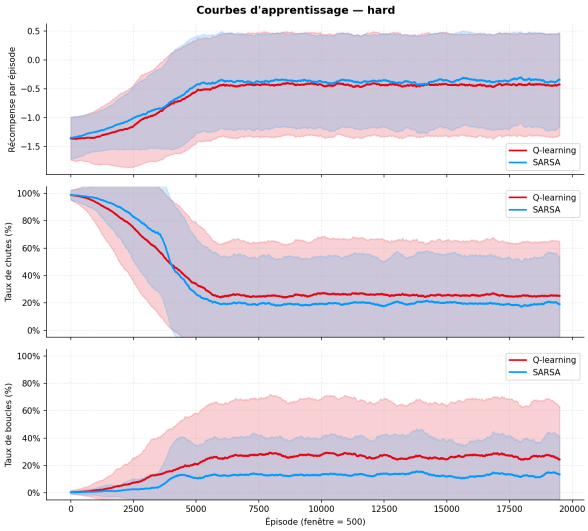
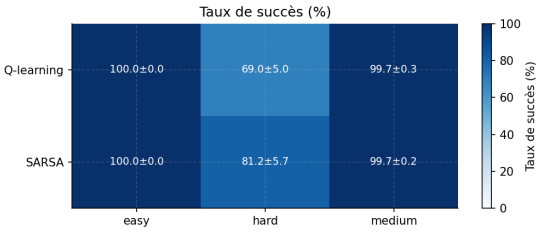
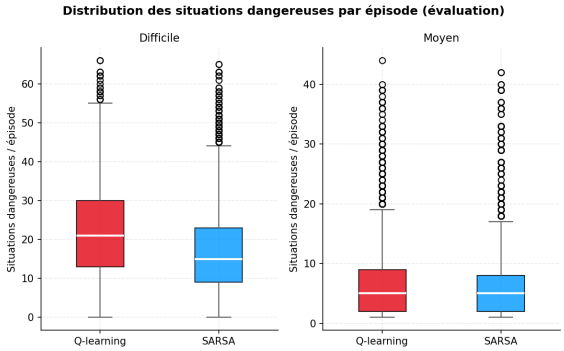


FIGURE 3 – **Difficile** — Les deux stratégies démarrent à un taux de chutes élevé, puis divergent : SARSA converge vers un taux de chutes et de boucles plus faible que Q-learning, reproduisant le phénomène de *cliff walking* [Sutton and Barto, 2018]. La mise à jour *on-policy* intègre le coût de l'exploration et oriente l'agent vers le détour sûr.



(a) Taux de succès à l'évaluation (politique greedy, 500 épisodes). Les échecs sont exclusivement des boucles (dépassements de la limite de pas) — le taux de chutes est nul.



(b) Distribution des situations dangereuses par épisode. Sur le cas Difficile, SARSA affiche une médiane plus basse et une variance moindre.

FIGURE 4 – Évaluation finale — comparaison Q-learning vs SARSA.

Le Tableau 2 synthétise les indicateurs d'évaluation. Sur le cas Difficile, la divergence est marquée : Q-learning accumule $31{,}0\% \pm 5{,}0\%$ de boucles contre $18{,}8\% \pm 5{,}7\%$ pour SARSA ($+12{,}2$ pts). SARSA réduit également la proximité aux trous ($25{,}9\%$ contre $29{,}9\%$) et emprunte nettement plus souvent le chemin sûr identifié par Dijkstra [Dijkstra, 1959] ($70{,}4\% \pm 2{,}9\%$ contre $60{,}1\% \pm 2{,}9\%$, écart de $+10{,}3$ pts), avec une variance identique confirmant la robustesse de cet avantage.

TABLE 2 – Indicateurs d'évaluation finale (politique greedy, 500 épisodes, moyenne $\pm$ écart-type, 10 répétitions). Chemin sûr : chevauchement $\geq 50\%$ avec le chemin Dijkstra (cas Difficile uniquement).

		Facile	**Moyen**	**Difficile**
Q-learning	Succès	$100{,}0 \pm 0{,}0\%$	$99{,}7 \pm 0{,}3\%$	$69{,}0 \pm 5{,}0\%$
	Boucles	$0{,}0 \pm 0{,}0\%$	$0{,}3 \pm 0{,}3\%$	$31{,}0 \pm 5{,}0\%$
	Danger	$20{,}0 \pm 0{,}0\%$	$28{,}5 \pm 1{,}6\%$	$29{,}9 \pm 2{,}4\%$
	Chemin sûr	—	—	$60{,}1 \pm 2{,}9\%$
SARSA	Succès	$100{,}0 \pm 0{,}0\%$	$99{,}7 \pm 0{,}2\%$	$81{,}2 \pm 5{,}7\%$
	Boucles	$0{,}0 \pm 0{,}0\%$	$0{,}3 \pm 0{,}2\%$	$18{,}8 \pm 5{,}7\%$
	Danger	$20{,}0 \pm 0{,}0\%$	$27{,}7 \pm 0{,}6\%$	$25{,}9 \pm 2{,}2\%$
	Chemin sûr	—	—	$70{,}4 \pm 2{,}9\%$

La Figure 5 visualise directement les politiques greedy apprises sur le cas Difficile. Les flèches révèlent que SARSA dirige systématiquement l'agent vers les colonnes droites (contour Dijkstra en vert) avant de descendre vers G, tandis que Q-learning présente des flèches contradictoires dans la zone centrale — symptôme de boucles dues à une sous-estimation du risque de glissade. Ce résultat illustre concrètement la distinction *off-policy*/*on-policy* : Q-learning, optimiste, accumule des trajectoires circulaires (31% de boucles) ; SARSA, conservateur, apprend un détour cohérent (18,8%).

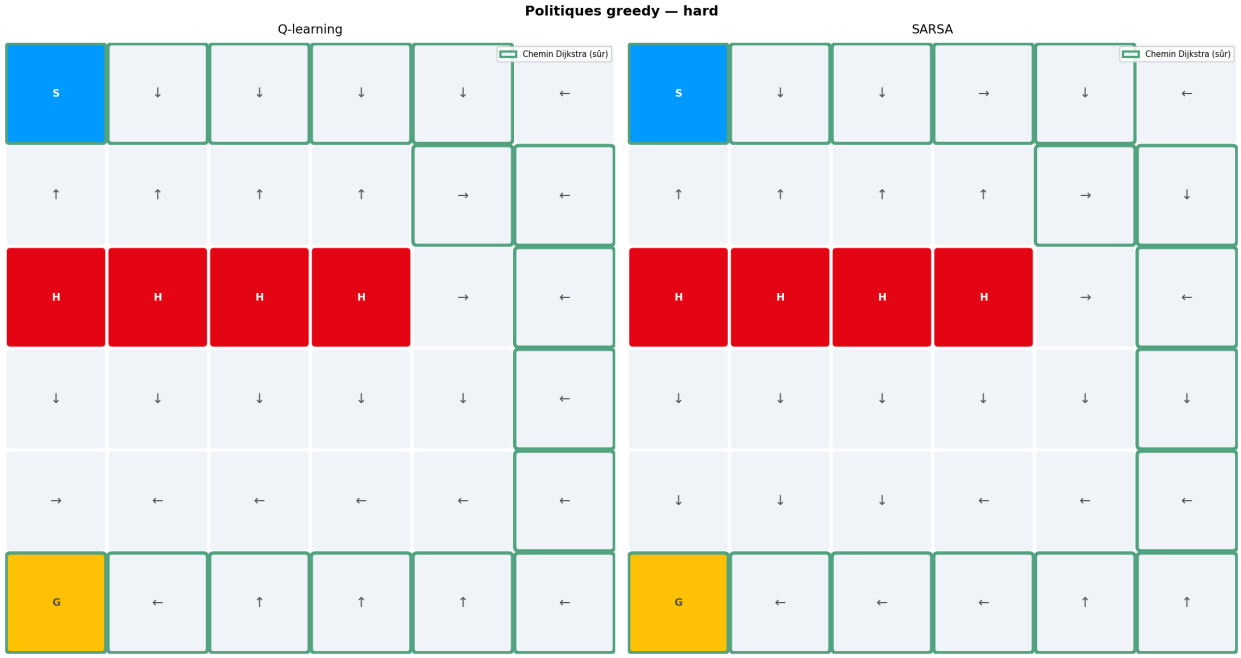


FIGURE 5 – Politiques greedy apprises sur le cas Difficile (moyenne sur 10 répétitions). Les flèches indiquent l'action greedy dans chaque état libre ; le contour vert délimite le chemin de référence Dijkstra. SARSA emprunte le détour sûr plus systématiquement que Q-learning.

En somme, Q-learning converge légèrement plus vite en début d'apprentissage mais au prix d'une instabilité persistante : plus de boucles (31% contre 18,8%), une plus grande proximité aux trous (29,9% contre 25,9%) et une politique qui emprunte moins souvent le détour sûr (60,1% contre 70,4%). SARSA, plus conservateur grâce à sa mise à jour *on-policy*, apprend un comportement plus sûr et plus cohérent. Ces observations soulignent les limites fondamentales des méthodes TD sans mécanisme explicite d'évitement du danger, et motivent l'introduction d'une telle approche dans le rapport final.

Références

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium : A standard interface for reinforcement learning environments. *arXiv preprint arXiv :2407.17032*, 2024.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning : An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540) :529–533, 2015.

Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4) :279–292, 1992.

Gavin A. Rummery and Mahesan Niranjan. On-line q-learning using connectionist systems. Technical Report CUED/F-INFENG/TR 166, University of Cambridge, 1994.

Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-Baselines3 : Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268) :1–8, 2021.

Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1 :269–271, 1959.